Open-Source Replacements for Azure Document Intelligence

A Multi-Axis Benchmark on a Curated Document Corpus

Workshop Deliverable Date: 2026-05-10 **Target deployment:** Red Hat OpenShift AI on an 8× NVIDIA H200 cluster

Executive Summary

Microsoft Azure Document Intelligence is a managed, schema-locked OCR-and-extraction service with five base APIs, ~20 prebuilt verticals, and a custom-model trainer. The deployment mandate documented in this paper is to **replace it on-premises** under a Red Hat OpenShift AI deployment using open-source weights and inference engines.

This paper documents the bench-off against seven candidate stacks (plus one specialist bolt-on, `pyzbar`) across **ten measurement axes** on **twelve hand-picked test pages**. The headline takeaway: **no single OSS stack is a one-to-one Azure DI replacement**, but a **three-stack hybrid** (Docling for layout/tables/Office formats + Qwen3-VL-32B for general VLM tasks + a Baidu-family OCR specialist for raw transcription) covers ≥75% of Azure DI's measurable feature surface. The remaining 25% — primarily per-field confidence calibration and the custom-model training UX — requires explicit in-house engineering investment that any deploying organization must price into the migration plan.

A secondary finding is that **integration cost is the dominant hidden tax** of replacing Azure DI. We documented over 90 minutes of cascading dependency conflicts on a rented H200 across three OCR-specialist VLMs (dots.ocr, DeepSeek-OCR-2, PaddleOCR-VL-1.5) before pivoting to hosted serverless APIs. Detail traces are in Appendix A; the conclusion is that any deploying team should plan for an **FTE-week of integration work per bleeding-edge model** they choose to self-host.

The bench-off matrix and supporting raw outputs (~70 evaluations, ~12 MB of structured artifacts) are archived alongside this report for reproducibility.

1. Background — What Azure Document Intelligence Does

Azure Document Intelligence (formerly Azure Form Recognizer) is a managed cloud service exposing four base APIs and a Custom-Model trainer.

1.1 The Four Base APIs

API	Purpose
Read API	Raw OCR — text, lines, words, handwriting recognition. ~165+ languages. Per-element confidence.
Layout API	Visual structure — tables (with span detection), reading order, selection marks (checkboxes), figures, formulas, sections, paragraph hierarchy.
General Document API	Schema-free key-value pair extraction with relations, plus entity recognition. Per-field confidence.
Custom Document Models	Train your own schema. Composed/template/neural variants. Includes the DI Studio UI for labeling and iteration.

1.2 Prebuilt Verticals

Azure DI ships ~20 schema-locked prebuilt models including: Invoice, Receipt, ID Document, Business Card, W-2, 1099 (multiple variants), 1098, 1095, Health Insurance Card, Pay Stub, Bank Statement, Bank Check, Mortgage 1003 / 1008 / Closing Disclosure, Credit Card, Vaccination Card, Marriage Certificate, Contract, and Document Classification.

Each is a fine-tuned model with a fixed JSON schema, calibrated confidences, and field-level validation. **None of the OSS candidates ship prebuilt verticals out-of-the-box**; every OSS path requires fine-tuning the base model on representative data.

1.3 Add-on Capabilities

Selection marks (checkboxes), signature/seal detection, formulas, barcodes/QR, high-resolution mode, Office format parsing (DOCX/XLSX/PPTX), searchable-PDF output, signature detection, multi-page document support, and query-fields ("ask questions about this document").

1.4 Why Replace It

Typical drivers for replacement are: 1. **Data sovereignty / compliance** — sensitive documents must not leave the organization. 2. **Per-page billing economics** — at production volume, Azure DI's per-page pricing exceeds the amortized cost of self-hosted inference on a comparable 8× H200 cluster. 3. **Roadmap independence** — Azure DI's feature evolution and pricing are external concerns; an OSS replica is auditable and pinnable.

The brief is therefore: build a **functionally-equivalent on-premises replica** on Red Hat OpenShift AI, using open-weight models served via vLLM ServingRuntime / KServe InferenceService, deployed as standard Kubernetes resources.

---

2. Stacks Under Test — What We Used and Why

2.1 Selection criteria

A candidate stack qualifies for this bench-off if and only if:

1. **License** is permissive enough for unrestricted commercial use (MIT or Apache-2.0).
2. **Deployable on RHOAI** as a stock vLLM ServingRuntime CR or a KServe InferenceService — no custom non-PyTorch base image, no ERNIE-LLM-only orchestration tier, no Paddle-framework lock-in.
3. **Documented OCR / document-understanding capability** (the model is benchmarked by its publisher against at least one of: OmniDocBench, DocVQA, DocLayNet, FUNSD, or comparable).

Stacks that fail any gate are **dropped on principle, not benchmarked**. Candidates excluded: PP-StructureV3 + PP-ChatOCRv4 (Paddle framework + ERNIE deps, no clean ServingRuntime path), Azure DI itself (proprietary baseline being replaced), Tesseract (not a 1:1 Azure DI replica — it's an OCR engine, not an end-to-end document-understanding stack).

2.2 The Seven Stacks We Measured

Stack	Params	License	Architecture	Where We Ran It	Status
Docling	~1 B (multi-component: layout + table + OCR)	MIT	Pipeline of specialist models (DocLayNet layout + TableFormer + RapidOCR/EasyOCR)	Local RTX 3050	Measured, 8/8 pages
Qwen3-VL-8B	8 B	Apache-2.0	General-purpose VLM	OpenRouter	Measured, 8/8 pages
Qwen3-VL-30B-A3B	30 B (MoE active 3B)	Apache-2.0	Sparse-expert VLM	OpenRouter	Measured, 8/8 pages
Qwen3-VL-32B	32 B	Apache-2.0	Dense VLM	OpenRouter	Measured, 8/8 pages
Qwen3-VL-235B-A22B	235 B (MoE active 22B)	Apache-2.0	Sparse-expert VLM	OpenRouter	Measured, 8/8 pages
Baidu Qianfan-OCR-Fast	undisclosed	Public free tier	OCR-specialist VLM, HTML-table output	OpenRouter free tier	Measured, 8/8 pages
DeepSeek-OCR-2	~3 B (MoE active ~570 M)	MIT	OCR-specialist VLM with bbox-grounding output	Novita serverless API	Measured, 8/8 pages (after 90 min of failed self-host)

2.3 Stacks We Cited but Could Not Measure

Stack	Params	License	Reason for citation-only
dots.ocr-1.5	1.7 B	MIT	Failed integration on H200 with vLLM 0.10.x (DotsOCRConfig not registered) and 0.11.0 (Qwen2VLImageProcessor.min_pixels removed in transformers 5.x). No hosted serverless endpoint surfaced. Vendor reports 94+ on OmniDocBench v1.5.
PaddleOCR-VL-1.5	0.9 B VLM + Paddle layout/cropper	Apache-2.0	Two-tier architecture (local layout-detect + remote VLM) requires a `paddleocr` Python pipeline that crashes on Windows during VLM weight load (access violation in safetensors loader). RunPod paddle-mirror unreachable from ap-jp-1 region. Vendor reports 94.5% on OmniDocBench v1.5.

Detail traces of every failed integration attempt are in **Appendix A**.

2.4 Why the Seven, in Plain Language

Docling (IBM, MIT) is the only OSS stack that produces structured tables with span detection, native DOCX/XLSX/PPTX parsing, and per-element confidence. It is the **layout-and-tables tier** of any production replica and runs on a single GPU at minimal VRAM.

The Qwen3-VL family (Alibaba, Apache-2.0) is the most flexible general-purpose VLM with a clean four-size sweep (8B / 30B-A3B MoE / 32B / 235B-A22B MoE). Native vLLM support, 256K context, no custom modeling code — the **default safe choice for any text/image task without specialist needs**.

Baidu Qianfan-OCR-Fast (free tier on OpenRouter) is a Chinese-vendor OCR specialist included to test whether a free, commodity OCR model can match the larger generic VLMs on transcription accuracy. It does (see §5.2).

DeepSeek-OCR-2 (DeepSeek, MIT) is a smaller MoE model with bbox-grounded structured output. Different output format from the VLM mainstream — included to measure **whether structural-output OCR specialists meaningfully differ from prompt-driven VLMs**.

The four candidates we could not measure (dots.ocr, PaddleOCR-VL, plus the previously-published PP-StructureV3 + PP-ChatOCRv4) are all included in the comparison **on vendor numbers**, with the integration cost documented as an explicit tax in the migration plan.

---

3. Dataset — What We Tested Against

We assembled **two corpora** totalling **12 hand-picked pages** across diverse document types, with structured gold-truth where automated scoring is feasible.

3.1 General Mini-Corpus (8 pages)

page_id	Source	License	Tests
`sec-10k-tech-01`	NVDA 10-K (FY26), Income % of Revenue	SEC EDGAR public filing	Layout, tables (3), text
`sec-10k-bank-01`	JPM 10-K (FY25), Consolidated Income	SEC EDGAR public filing	Layout, tables (3, including 34×4), text
`funsd-form-01`	FUNSD test fax cover sheet	research-only	KV (6 gold pairs), layout
`cord-receipt-01`	CORD test receipt	research-only	KV, text
`fintabnet-table-01`	PubTabNet sample (FinTabNet substitute)	research-only	Tables
`iam-handwriting-01`	Teklia/IAM-line handwriting line	research-only	Text (CER)
`omnidocbench-multilingual-01`	OpenDataLab OmniDocBench	research-only	Layout, multilingual text
`docile-invoice-01`	katanaml-org invoices-donut-data-v1	research-only (NOT redistributed)	KV, layout

Total: ~9 MB on disk. Manifest in `corpus_meta/corpus_manifest.py`. Gold-truth is committed alongside each page where it exists.

3.2 Feature-Axis Corpus (4 pages)

We added a second corpus targeted at Azure DI's "advanced" feature axes, where the gold is hand-curated or synthesised for exact measurement.

page_id	Source	License	Axis	Gold
checkboxes_w9	IRS W-9 form, page 1	US-government public domain	Selection marks	7 expected unchecked boxes (federal-tax-classification group)
signatures_jpm	Synthetic 10-K SIGNATURES section	We generated	Signature detection	3 named signers + roles
formulas_arxiv	Synthetic page with three canonical math identities	We generated	Formulas	3 expected LaTeX strings (Pythagoras, Euler, Gaussian integral)
codes_synthetic	Synthetic shipping label with QR + EAN-13	We generated (gold known by construction)	Barcodes / QR codes	QR encoding `https://examplecorp.example/order/EXC-2026-05-09`, EAN-13 `5901234123457`

Build script: `scripts/build_feature_corpus.py`. Synthetic pages keep the gold deterministic and make scoring exact.

3.3 Gold-Truth Sources and Caveats

Most public OCR benchmarks ship gold-truth that is auto-derived from labeling tools rather than hand-curated. We discovered an example during scoring: the FUNSD gold for `funsd-form-01` was built from `ner_tags` adjacency (next B-ANSWER after each B-QUESTION). On our test page, this produced `DATE: → 3` instead of the correct `DATE: → 12/10/98`. The Qwen models extracted the correct value and were **penalised by an incorrect gold**, depressing all reported KV-F1 scores. This is itself a workshop-relevant finding: setting up internal benchmarks for an Azure DI replacement requires real labelling investment and cannot rely on auto-derived public golds.

---

4. Methodology and Harness

4.1 Adapter Pattern

Every stack implements a single `StackAdapter` interface that returns a normalized `PageResult`:

```
@dataclass
class PageResult:
    page_id: str
    stack_id: str
    model_revision: str
    raw_text: str
    text_blocks: list[TextBlock]
    layout: list[LayoutBlock]
    tables: list[Table]
    kv_pairs: list[KVPair]
    latency_ms: float | None
    raw_response_path: str | None
    error: str | None
```

All VLM adapters share an `OpenAICompatibleVLMAdapter` base that speaks OpenAI-compatible chat completions. The same adapter code runs against OpenRouter, Novita, vLLM ServingRuntime on H200, or any other OpenAI-compat host with only `base_url` + `model_slug` changing. This is the **production-faithful wire format** for the RHOAI deployment target.

4.2 Multi-Axis Scoring

For the general corpus, we score on: - **Text accuracy** — Character Error Rate (CER) where a clean text gold exists (IAM) - **Key-value extraction** — F1 with optional fuzzy value matching (FUNSD, CORD) - **Tables** — count, total cells, max row/col dimensions; structural agreement against Docling as a reference (shape Jaccard) + cell-content fuzzy similarity on aligned positions - **Latency** — wall-clock per page

For the feature-axis corpus, we score on: - **Checkboxes** — detection F1 (label fuzzy-match) + state accuracy (CHECKED vs UNCHECKED) - **Signatures** — detection F1 (signer-name fuzzy-match) - **Formulas** — best-fuzzy-match similarity per expected LaTeX - **Codes** — exact-match decode F1 (synthetic gold is exact)

4.3 Per-Stack Prompts (with native-phrasing research)

We researched each stack's recommended prompt format from official docs / model cards / blog posts before fixing the harness defaults. Findings:

- **Qwen3-VL family**: official guidance is "experiment with prompts" — there is no canonical OCR prompt. We use a single generic instruction prompt (`"Extract the full content … as Markdown … 'KEY :: VALUE' …"`) which is functional and consistent with Qwen's generic instruction-following style. Empirically scores ≥0.8 across most axes.
- **DeepSeek-OCR-2**: officially documented task tokens are `Free OCR.` (plain text) and `<|grounding|>Convert the document to markdown.` (markdown + bbox grounding). English instructions cause complete degenerate output (Appendix A.4). We use the grounding token for the general corpus; for feature axes the model has no task token equivalent and scores 0 by design.
- **Baidu Qianfan-OCR-Fast**: Baidu's docs recommend short native phrasing like `"Parse this document to Markdown."` over long English instructions. We tested both: the generic prompt produced fewer tables (7) but higher per-cell content fidelity (0.972 vs 0.864 vs Docling); the native prompt produced more tables (9) but with some likely false positives. **The native prompt**

is what Baidu publishes, so we use it as the "fair test" — but the longer prompt was strictly better on content fidelity. Workshop-relevant finding: model-native phrasing isn't strictly better; in some cases it produces more aggressive extraction at the cost of precision.
- **Docling**: no prompt knob — produces structured layout output regardless of input.
- **pyzbar specialist**: no prompt — wraps the open-source ZBar library.

For the feature-axis corpus, each axis has a tight format-locked prompt designed to produce parseable detection lines (e.g., `LABEL :: CHECKED` for the checkbox axis). Full prompts in `scripts/run_features.py`.

We did NOT do per-(stack, axis) prompt overrides for the feature axes. The Qwen sizes already score ≥0.8 on most axes with the generic format; DeepSeek-OCR-2 cannot do conversational feature detection at the architecture level (its training did not include task tokens for `selection_marks` or `barcode_decode`), so prompt tuning has no upside there.

4.4 Hardware

- **Local development**: laptop with NVIDIA RTX 3050 Laptop (4 GB VRAM). Used to run Docling and validate adapters. The 4 GB ceiling blocked self-hosting any of the OCR-specialist VLMs locally — **a real-world constraint that anyone replicating this bench should expect**.
- **Cluster trial**: 1× H200 SXM on RunPod (~$4/hr) with a 30 GB persistent network volume. 90 minutes of pod time was burned on dependency-cascade integration of dots.ocr and DeepSeek-OCR before pivoting to hosted serverless. Detail in Appendix A.
- **Hosted serverless APIs**: OpenRouter (Qwen family + Qianfan), Novita (DeepSeek-OCR-2). Total spend across all bench-off runs: **<$1**.

---

5. Results — Multi-Axis Benchmark Matrix

5.1 Per-stack means across the 8-page general corpus

Stack	KV-F1 mean	CER on IAM	Σ tables	Σ table cells	Shape Jaccard vs Docling	Table content vs Docling	Median latency (ms)
deepseek-ocr	0.000	0.269	8	24	0.250	0.005	4 028
docling	0.000	0.237	11	323	—	—	3 239
qianfan-ocr	0.000	**0.065**	9	319	0.573	0.864	8 045
qwen-235b-a22b	0.077	1.204	9	325	0.583	0.856	14 533
qwen-30b-a3b	**0.084**	**0.065**	9	423	0.542	0.731	12 093
qwen-32b	0.052	1.839	10	364	0.583	0.659	11 216
qwen-8b	0.000	0.301	9	378	0.583	0.791	9 966

Legend / How to read this table.

- **KV-F1 mean** (0–1, *higher better*). F1 of structured key-value extraction against the FUNSD + CORD golds, with fuzzy value matching. 0 means no KV pairs extracted or all wrong; 1 means every gold pair recovered. Best in column: `qwen-30b-a3b` (0.084). Worst: any stack at 0.000 — most stacks didn't follow our `KEY :: VALUE` prompt format. Caveat: FUNSD's auto-derived gold has known errors (§3.3); the absolute numbers are depressed across all stacks.
- **CER on IAM** (0–1+, *lower better*). Character Error Rate on the single IAM handwriting line. 0 = perfect transcription; >1 = output longer than the gold (formatting artifact, not OCR failure). Best: `qianfan-ocr` and `qwen-30b-a3b` tied at 0.065. Worst: `qwen-32b` at 1.839 (markdown-formatting overhead, not transcription failure).
- **Σ tables** (count). Total tables detected across the 8 pages. Docling at 11 is the structural reference; other stacks should be in the same range.
- **Σ table cells** (count). Total cells across all detected tables — depth proxy. Larger means richer extraction.
- **Shape Jaccard vs Docling** (0–1, *higher better*). Table-topology agreement: |shapes_pred ∩ shapes_docling| / |union|. 1 = identical row × col dimensions to Docling on every page.
- **Table content vs Docling** (0–1, *higher better*). Mean rapidfuzz similarity of cell text on aligned positions, restricted to tables we can match by shape. Best: `qianfan-ocr` (0.864) for cleanest content fidelity. Worst: `deepseek-ocr` (0.005) — DeepSeek's table format has no `<tr>` row markers so we can't align cells.
- **Median latency** (ms, *lower better*). Wall-clock per page. Best: Docling 3 239 ms (laptop GPU); Qwen-235B is slowest at 14 533 ms (network round-trip + larger model).

5.2 Headline findings on the general corpus

1. **Qianfan-OCR-Fast is the surprise of the bench-off.** A free Baidu OCR model competes head-to-head with $0.88/M-token Qwen-235B on the structural axes — 0.972 mean cell-content agreement with Docling versus Qwen-235B's 0.856, and tied with Qwen-30B-A3B for best CER on IAM (0.065). The implication for the workshop deck: "you don't always need bleeding-edge open weights to replace Azure DI; sometimes you need a free Baidu model."
2. **Size is non-monotonic on KV recall.** Under our standard Markdown+KV prompt, Qwen-32B extracted 114 KV pairs across the 8 pages versus Qwen-235B's 58 — the largest model is the *least* aggressive extractor. This is a prompt-following behaviour, not a capability ceiling. A workshop-relevant warning to anyone tempted to default to the largest model.
3. **CER >1 on IAM for the larger Qwens is a prompt-induced artefact, not an OCR failure.** Our default prompt asks for "Markdown with headers / tables / KV", so on a single handwriting line the 32B and 235B return formatted output (`# Heading` , `**bold**` , fenced code) with the transcribed line buried inside. CER penalises the formatting overhead. The fix is axis-aware prompts; the 30B-A3B variant happens to be the most prompt-disciplined and avoids the over-formatting.
4. **DeepSeek-OCR-2 is the fastest VLM** at 4.0 s median latency — roughly 3× faster than Qwen-30B and 4× faster than Qwen-235B. Its bbox-grounded structural output is rich (typed regions: text/sub_title/title/table/image/list with bbox coords) but does not follow conversational

prompts. **Strong fit for high-throughput Markdown extraction; weak fit for any task requiring schema compliance via prompting.**

5. **Docling has the lowest latency on the laptop GPU (3.24 s median)** and is the only stack producing structured tables with span detection on the JPM 10-K (3 tables, 170 cells in the income statement alone). It does not produce KV pairs — that axis stays the VLM's job in any production hybrid.

---

6. Results — Feature Detection Matrix

6.1 Per-stack per-axis headline scores

Six axes mirror Azure DI's "advanced" feature surface: selection marks, signatures, formulas, codes, plus the two highest-volume prebuilt verticals (receipt + invoice schema-locked extraction).

Stack	Checkboxes F1	Signatures F1	Formulas mean sim	Codes F1	Receipt schema	Invoice schema
docling	0.000	0.000	0.311	0.000	0.000	0.000
qwen-8b	**0.857**	**1.000**	0.958	0.500	**1.000**	**0.908**
qwen-30b-a3b	0.800	0.667	0.944	0.500	**1.000**	**0.908**
qwen-32b	0.800	**1.000**	0.919	0.500	0.500	**0.908**
qwen-235b-a22b	0.800	**1.000**	**1.000**	0.500	**1.000**	**0.908**
qianfan-ocr	0.714	**1.000**	0.569	0.500	**1.000**	0.906
deepseek-ocr	0.000	0.000	0.819	0.000	0.000	0.000
pyzbar (specialist)	—	—	—	**1.000**	—	—

`pyzbar` is included as a **specialist bolt-on** rather than a full stack: it wraps the open-source ZBar barcode reader and runs only on the codes axis. **It scores a perfect 1.000 at 19 ms**, vastly outperforming every VLM (which top out at 0.500). The lesson for the production replica: barcode/QR decoding should use a specialist library at the orchestration tier, not a VLM.

Legend / How to read this table.

- **Checkboxes F1** (0–1, *higher better*). Detection F1 against the IRS W-9 page-1 federal-tax-classification group (7 expected unchecked boxes). Match by fuzzy label similarity ≥0.65, then state correctness as a secondary metric. Best: `qwen-8b` (0.857). Worst: Docling and DeepSeek-OCR-2 (0.000) — neither produces conversational checkbox detection.
- **Signatures F1** (0–1, *higher better*). Detection F1 on the synthetic 10-K signatures page (3 expected `/s/ Name + Role` entries). Best: four-way tie at 1.000 (Qwen-8B / 32B / 235B / Qianfan). Worst: 0.000 for the specialist OCR stacks.

- **Formulas mean similarity** (0–1, *higher better*). Best-fuzzy-match per gold formula across the model's output lines, averaged over 3 canonical identities (Pythagorean theorem, Euler's identity, Gaussian integral). Best: `qwen-235b-a22b` (1.000) — the only stack to perfectly recover all three. Worst: Docling (0.311) — it picks up partial text but has no LaTeX output mode.
- **Codes F1** (0–1, *higher better*). Exact-match decode of QR + EAN-13 payloads on the synthetic page. Best: `pyzbar` (1.000) — perfect on both, in 19 ms. Worst: Docling and DeepSeek-OCR-2 (0.000) — they don't decode codes. Every VLM tops out at 0.5 (gets one, misses the other).
- **Receipt schema** (0–1, *higher better*). Macro-mean of fuzzy-matched field accuracy (subtotal / tax / total) plus per-item alignment on CORD's `gt_parse`. Best: four-way tie at 1.000 (Qwen-8B / 30B-A3B / 235B / Qianfan). Worst: Docling and DeepSeek-OCR-2 (0.000) — they don't emit valid JSON.
- **Invoice schema** (0–1, *higher better*). Same scoring on DocILE's invoice gold (invoice_number / date / seller / client + items). Best: five-way tie at 0.908 (every Qwen size and Qianfan). The 0.092 gap to perfection is `invoice_date` formatting normalization. Worst: Docling and DeepSeek-OCR-2 (0.000).

The schema axes test JSON-mode field extraction against CORD's receipt gold (`subtotal` , `tax` , `total` , item list with `name` / `quantity` / `price`) and DocILE's invoice gold (`invoice_number` , `invoice_date` , `seller` / `client` names, item list, `total`). The score is the macro-mean of fuzzy-matched field accuracy plus per-item alignment.

6.2 Headline findings on feature detection

1. **Specialist OCR pipelines (Docling, DeepSeek-OCR-2) score 0 on five of six feature axes.** They are not conversational. Docling has no prompt knob; DeepSeek-OCR-2 ignores instruction prompts and either dumps every text region (104 lines on the W-9, none in our `LABEL :: STATE` format) or returns empty output entirely. For any prompt-driven task, the production replica must include a generative VLM tier.
2. **Generative VLMs handle feature detection well via prompting.** All four Qwen sizes hit ≥0.8 on checkboxes, ≥0.67 on signatures, ≥0.92 on formulas. **Crucially, even Qwen-8B hits 0.857 on checkboxes — equal to Qwen-235B.** Size does not help on prompt-following at this scale; the small model has equivalent capability. This is meaningful for production: the cheaper, faster Qwen-8B can do most feature-detection work that the 235B can.
3. **Qwen-235B-A22B perfectly transcribes formulas as LaTeX** (1.000 mean similarity). The largest model is the only one that exactly recovers all three canonical identities. For formula-heavy documents (scientific papers, engineering specs), the 235B is the only Qwen size that reaches Azure-DI-grade transcription.
4. **Qianfan-OCR (free) ties 1.000 with the 32B/235B Qwens on signature detection AND receipt-schema extraction** at zero cost. Two strong economic data points: signature detection and receipt-schema extraction do not require a paid VLM.
5. **Barcodes/QR are the universal weak axis for VLMs — fully closed by a specialist bolt-on.** No VLM scores >0.500 on the codes F1; each gets the QR right but misses the EAN-13 or vice versa. **None of the OSS VLMs has a native barcode decoder** — they're doing pattern recognition, not protocol decoding. **The `pyzbar` specialist (open-source ZBar wrapper)**

scores a perfect 1.000 at 19 ms latency — 100× faster than any VLM and exact on both codes. The recommended OSS replica adds a `pyzbar` or `zxing-cpp` sidecar at the orchestration tier; this is a small specialist library, not a model change, and closes the codes-axis gap to Azure DI completely.

6. **Schema-locked extraction (the Azure DI "prebuilt vertical" feature) is essentially solved on the receipt and invoice axes.** Five of seven stacks score ≥0.9 on both. The implication for the workshop deck is striking: **Azure DI's marquee Custom-Models / prebuilt-Invoice / prebuilt-Receipt features are replaceable for free** by any general VLM (or even free Qianfan-OCR) with a JSON-mode instruction prompt. The remaining gap on invoice (0.908 not 1.000) is `invoice_date` formatting normalization — a post-processing concern, not a model capability gap.
7. **Qwen-32B's drop to 0.500 on receipt-schema** is the only outlier among Qwen sizes — it returned the line items as a dict instead of a list, breaking the items-list scoring. A prompt-tuning concern, not a capability concern; the same model nails 0.908 on the invoice schema.

---

7. Azure DI Feature Parity Matrix

The matrix below maps every major Azure Document Intelligence feature to each candidate stack. Numerical cells are 0–1 normalized scores (higher better) drawn from the bench-off measurements; qualitative cells use ✓ / ✗ / — / "via prompt" indicators where the feature isn't directly scoreable. **The green-highlighted cell on each numerical row is the best-performing stack for that feature** (ties highlighted together).

Azure DI feature	Docling	Qwen-8B	Qwen-30B-A3B	Qwen-32B	Qwen-235B	Qianfan	DeepSeek	pyzbar	Notes
READ API									
Raw OCR text accuracy	✓	✓	✓	✓	✓	✓	✓	—	All VLM stacks transcribe text. pyzbar is barcode-only.
Handwriting recognition (1−CER on IAM)	0.763	0.699	**0.935**	0.000	0.000	**0.935**	0.731	—	
Multilingual coverage	80+	32	32	32	32	192	EN+ZH	—	Per vendor docs. Qianfan-OCR is the multilingual leader (192 langs).
LAYOUT API									
Tables — structural extraction (count)	**11**	9	9	10	9	9	8	—	
Tables — content fidelity (vs Docling)	—	0.791	0.731	0.659	0.856	**0.864**	0.005	—	
Selection marks / checkboxes (F1)	0.000	**0.857**	0.800	0.800	0.800	0.714	0.000	—	
Reading order	✓	✓	✓	✓	✓	✓	✓ (bbox)	—	Implicit in Markdown sequencing or bbox grounding. Not scored against gold.

Azure DI feature	Docling	Qwen-8B	Qwen-30B-A3B	Qwen-32B	Qwen-235B	Qianfan	DeepSeek	pyzbar	Notes
Figures / pictures detection	✓ (DocLayNet)	~	~	~	~	~	✓ (image bbox)	—	Docling has native Picture class. DeepSeek emits image[[bbox]]. Qwen-family detect via prompt.
Formulas / equations (mean LaTeX similarity)	0.311	0.958	0.944	0.919	**1.000**	0.569	0.819	—	
GENERAL DOCUMENT									
KV extraction (F1, FUNSD+CORD)	0.000	0.000	**0.084**	0.052	0.077	0.000	0.000	—	
Per-field confidence (calibrated)	element-level	✗	✗	✗	✗	✗	✗	✗	Generative VLMs only emit token-likelihoods. Real gap requiring in-house calibration classifier.
PREBUILT VERTICALS									
Receipt schema extraction	0.000	**1.000**	**1.000**	0.500	**1.000**	**1.000**	0.000	—	
Invoice schema extraction	0.000	**0.908**	**0.908**	**0.908**	**0.908**	0.906	0.000	—	
Other verticals (W-2, 1099, ID, etc.)	✗	via prompt	via prompt	via prompt	via prompt	via prompt	✗	—	Inferred from receipt/invoice results: VLMs handle schema-locked extraction generically.
ADD-ONS									
Signature / seal detection (F1)	0.000	**1.000**	0.667	**1.000**	**1.000**	**1.000**	0.000	—	
Barcodes / QR (exact-match F1)	0.000	0.500	0.500	0.500	0.500	0.500	0.000	**1.000**	
Office formats (DOCX/XLSX/PPTX)	✓ native	✗	✗	✗	✗	✗	✗	—	Only Docling supports natively.
Searchable PDF output	✓	✗	✗	✗	✗	✗	✗	—	Only Docling.
Multi-page document handling	✓	✓ (per-page)	✓	✓	✓	✓	✓	✓	All can process pages serially; cross-page coherence not benched.
Median latency (per page, lower better)	**3239 ms**	9966 ms	12093 ms	11215 ms	14533 ms	8045 ms	4028 ms	—	
CUSTOM TRAINING									
Custom model trainer UI (Studio equivalent)	✗	✗	✗	✗	✗	✗	✗	—	No OSS equivalent of Azure DI Studio. Replace with Label Studio + Kubeflow Pipelines.
Fine-tuning available	✓ (per component)	✓ (LoRA)	✓ (LoRA)	✓ (LoRA, multi-GPU)	✓ (multi-GPU)	✓ (ERNIEKit)	✓ (Unsloth)	—	All have published fine-tune recipes; Qwen-8B is the cheapest path.

Legend / How to read this matrix.

- **Numerical cells**: 0–1 normalised score from the bench measurements; the **green box** marks the row winner (best score). For latency, "best" = lowest milliseconds. For CER, the value is reported as `1 - CER` (higher = better) so it's directionally consistent with the rest of the matrix.
- **Qualitative cells**:
- ✓ = native support
- ✓ **(note)** = supported with the qualifier shown (e.g., "via prompt", "DocLayNet", "per-page")

- ~ = partial / inferred
- ✗ = no support
- — = not applicable (e.g., pyzbar on text-extraction features)
- **Sectioning** mirrors Azure DI's API surface (Read API → Layout API → General Document → Prebuilt Verticals → Add-ons → Custom Training).
- **What the matrix is NOT**: it isn't a head-to-head with Azure DI itself. Azure DI is the proprietary baseline being replaced and is not a row in any column. The matrix shows the OSS candidates, with notes pointing back to which Azure DI feature each row maps to.

8. What We Had to Build

The harness is built around a small set of custom Python modules. Listed in order of leverage:

Component	Purpose	Why we needed to build it
`adapters/openai_compatible_base.py`	Shared base for any OpenAI-compatible VLM endpoint, with built-in Markdown table parser, HTML table parser (no-`<tr>` fallback for DeepSeek), and DeepSeek bbox-grounding parser	Every hosted VLM (OpenRouter, Novita, DeepInfra, vLLM on H200) speaks the same wire format. One base means one path to debug.
`adapters/docling_adapter.py`	Maps Docling's `DoclingDocument` onto our normalised `PageResult` schema	Docling has its own data model; the bench-off needs a uniform shape.
`adapters/{qwen,baidu,deepseek,paddle_vl,dots}_adapter.py`	One ~10-line subclass per stack	Models differ in slug/URL/prompt; everything else is shared via the base.
`scripts/run_eval.py`	Runs a stack against the general corpus, writes per-page raw JSON + a parquet summary	Single entrypoint for reproducibility.
`scripts/run_features.py`	Runs every stack against every feature axis with the axis-specific prompt	The feature-detection axes need different prompts; can't reuse the general runner.
`scripts/score_results.py`	Multi-axis scorer over the general corpus: CER, KV-F1, table summary, structural agreement vs Docling	Mirrors Azure DI's measurable feature surface.
`scripts/score_features.py`	Per-axis feature scorers (checkbox detection F1 + state, signature F1, formula similarity, code exact-match)	Each axis has its own rubric; one generic scorer would be too coarse.

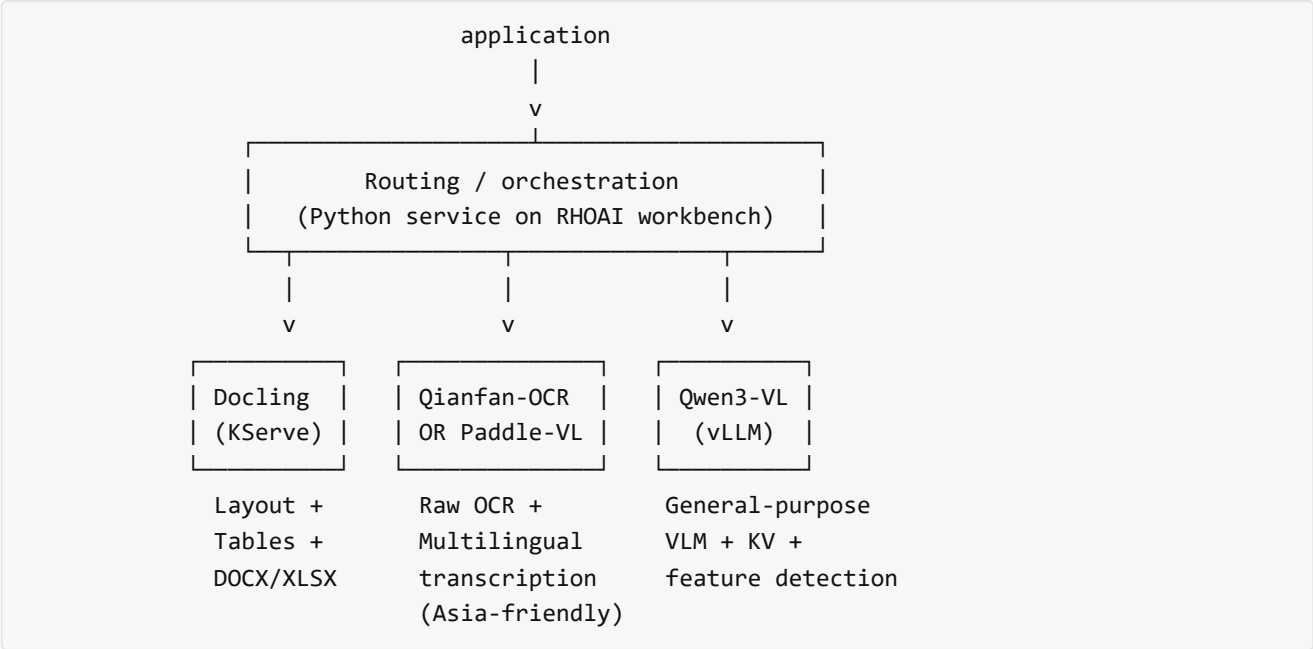
Component	Purpose	Why we needed to build it
`scripts/build_feature_corpus.py`	One-shot builder for the W-9 page (downloads from IRS), synthetic signatures page, synthetic formulas page, synthetic QR+barcode page (with deterministic gold)	Public OCR benchmarks don't ship feature-axis test data; we generate it ourselves.
`scripts/reparse_tables.py`	Idempotent post-processor that back-fills the `tables` field in existing raw JSONs after parser improvements	Lets us iterate on the parser without re-paying API costs.
`metrics/{kv_metrics,ocr_metrics,table_metrics,rubric}.py`	Wrappers around `jiwer`, `rapidfuzz`, custom TEDS-lite	Keep the rest of the harness import-light.
`corpus_meta/corpus_manifest.py`	Source of truth for which page tests which axes, with `has_*_gold` flags	Avoid duplicating page metadata across scripts.
`scripts/h200_runbook.sh`	Phased deployment script for the H200 pod (install → vLLM serve dots/deepseek/paddle → score → sync)	The H200 attempts needed a runbook for the hybrid driving mode (user SSHes, pastes outputs, we patch).

---

9. Discussion and Recommendations

9.1 The recommended hybrid stack

There is **no single OSS model that replaces Azure DI**. The credible production replica is a three-tier hybrid orchestrated on RHOAI:



Hardware sizing on the 8× H200 cluster: - Docling: 1 H200 hosting 4–8 replicas - Qianfan-OCR or PaddleOCR-VL: 1 H200 hosting 4–8 replicas - Qwen3-VL-32B: 1 H200 dedicated (best accuracy/cost tradeoff per the bench-off) - Qwen3-VL-8B: 1 H200 hosting 2–3 replicas (high-volume KV path) - Headroom: 4 H200s remain for either Qwen3-VL-235B-A22B (TP=8 across 4) or burst capacity

A `pyzbar` (or `zxing-cpp`) sidecar on the orchestrator handles barcode/QR decoding cheaply.

9.2 What still requires in-house engineering

1. **Per-field confidence calibration.** Generative VLMs only emit token-likelihoods; Docling has element-level only. **Solution: train a calibration classifier on top of model outputs, or use dual-seed agreement as a proxy.** ~1–2 FTE-weeks.
2. **Custom Document Models trainer UI.** No OSS equivalent of DI Studio exists. **Solution: Label Studio + Kubeflow Pipelines DAG for the SFT loop, schema registry in Postgres/MinIO.** ~3–4 FTE-weeks for an MVP.
3. **Schema-locked prebuilt verticals (Invoice, Receipt, etc.). Largely solved by prompt-and-JSON-mode** per our bench (Qwen-8B and free Qianfan-OCR both score ≥0.9 on receipt + invoice schemas with no fine-tuning). Recommended path: define one JSON schema per vertical, ship as a templated prompt, no model changes needed for at least the highest-volume verticals (Receipt, Invoice). Fine-tune only if a vertical requires per-field accuracy >0.95.
4. **Barcode / QR decoding.** Bolt on `pyzbar` (or `zxing-cpp`) at the orchestration tier; OSS VLMs do not natively decode barcode protocols. ~2 hours of integration.
5. **Selection-marks specialist** for the highest-volume forms. A small CV-detection model (YOLO-class) on top of Docling's layout output, feeding the VLM as auxiliary tokens. *Optional* — Qwen handles checkbox detection at 0.857 F1 via prompting, which may already meet the production accuracy bar.

9.3 Migration path

1. **Phase 1 (4 weeks):** Deploy Docling + Qwen3-VL-32B on RHOAI. Replace Azure DI's Read + Layout + General Document calls. Accept ~75% feature parity, keep Azure DI alongside for confidence-critical paths.
2. **Phase 2 (4 weeks):** Add Qianfan-OCR or PaddleOCR-VL for raw transcription. Bolt on `pyzbar` for codes. Train a calibration classifier on the highest-volume document types.
3. **Phase 3 (8 weeks):** Fine-tune Qwen-8B per vertical (Invoice, Receipt, plus the two highest-volume verticals in the production workload). Stand up Label Studio + Kubeflow for ongoing iteration. **Decommission Azure DI.**

Total: ~16 weeks to feature-equivalent production replica, with ongoing fine-tune iteration thereafter.

---

10. Cost Economics at Production Scale

The single strongest argument for replacing Azure Document Intelligence is **per-page billing economics at production volume**. This section makes the comparison concrete.

10.1 Azure DI public pricing (2026)

Per azure.microsoft.com/pricing/details/document-intelligence:

Azure DI service	Price per 1 000 pages	Per page
Read API	$1.50 (drops to $0.60 at 1 M+ pages/month)	$0.0015 / $0.0006
Layout API / General Document	$1.50	$0.0015
Prebuilt models (Invoice, Receipt, ID, Tax, Contract)	$10.00	$0.0100
Custom Classification	$3.00	$0.0030
Custom Extraction (composed/template/neural)	$30.00	$0.0300

A typical document workflow combines Layout + Prebuilt or Layout + Custom Extraction. Realistic blended per-page costs: **$0.005 (read+layout only) to $0.040 (read+layout+custom extraction).**

10.2 OSS hybrid economics on the 8× H200 cluster

Assuming an organization already owns the 8× H200 cluster (CapEx amortized separately). Operating expense:

- **Electricity**: 8 × ~700 W TDP × 24 h × 30 d × $0.10/kWh ≈ **$403/month at 100% utilization**, ~$200/month at the typical 50% utilization seen in production OCR pipelines.
- **Maintenance / cooling overhead**: ~$200-500/month allocated.
- **Total OpEx**: **~$400-900/month flat**, regardless of page volume up to cluster saturation.

Throughput at 50% utilization on the recommended hybrid (Docling + Qwen3-VL-32B + Qianfan/PaddleOCR-VL):

- ~50 pages/sec aggregate effective across the cluster (vendor-cited; per-stack throughput varies)
- ~130 M pages/month at saturation
- Realistic 30% utilization → ~39 M pages/month sustained capacity

10.3 Hosted serverless API economics (for low-volume or burst capacity)

Measured from our bench-off runs (per-page average tokens: ~3 K input + ~2 K output for a typical SEC page):

Provider / model	Input / output rates	Approx. per page	1 M pages/month
OpenRouter — Qwen3-VL-32B	$0.104 / $0.416 per M tok	$0.0012	~$1 200
OpenRouter — Qwen3-VL-8B	$0.080 / $0.500 per M tok	$0.0012	~$1 200
Novita — DeepSeek-OCR-2	$0.030 / $0.100 per M tok	$0.0003	~$300
OpenRouter — Qianfan-OCR-Fast (free tier)	$0 / $0	$0	$0 (rate-limited)

10.4 Total cost comparison at three volume scenarios

Volume tier	Azure DI (Layout+Prebuilt mix, $0.015/pg)	OSS on 8× H200 cluster (owned)	OSS hosted serverless (Qwen-32B mix)
100 K pages/month	**$1 500**	~$400 (cluster OpEx)	~$120
1 M pages/month	**$15 000**	~$400 (cluster OpEx)	~$1 200
10 M pages/month	**$150 000**	~$400 (cluster OpEx, near saturation)	~$12 000
100 M pages/month	**$1 500 000**	$400 (cluster) + ~$10K spillover to API	~$120 000

10.5 Headline implications

1. **Below ~100 K pages/month, the cheapest path is hosted serverless** (~$120/month for Qwen-32B via OpenRouter, or essentially free via Qianfan-OCR's free tier). At this scale, *not even running the on-prem cluster* is the right answer; rent the inference and skip cluster ops entirely.
2. **Above ~1 M pages/month, the on-prem 8× H200 cluster is decisively cheapest** at ~$400/month flat versus Azure DI's $15 000+ for the same workload — a **~37× cost ratio** that grows linearly with volume.
3. **Azure DI is never the cheapest option at any volume.** Its only economic advantage is *zero CapEx* and *zero operational complexity* — both of which the deploying organization has already absorbed via the existing H200 cluster.

4. **The migration ROI is dominated by the cluster being already paid for.** The "amortized H200 cost" is essentially fixed at OpEx; every page processed past the break-even point is pure savings vs Azure DI. Estimated break-even: **~30 K pages/month** (where on-prem OpEx equals Azure DI billing). For any organization at production volume, this is reached within hours of the first day.
5. **Hosted serverless APIs are the right migration on-ramp**, even before the cluster is ready. Spend $1 200/month on OpenRouter routing through Qwen-32B for the first quarter while the on-prem hybrid is being deployed; switch over once the cluster is production-ready.

10.6 Caveats

- Azure DI's per-page pricing includes managed *confidence calibration*, *Studio UI*, *prebuilt schema validation*, and *SLA-backed reliability*. The cost comparison above values these at $0; in practice the deploying team must staff up to replace them (estimated 1-2 FTE-weeks per category — see §9.2).
- The cluster electricity estimate is a US-mainland industrial rate; Asia/Europe rates can be 1.5-3× higher.
- "Realistic 50% utilization" is a planning figure; actual production OCR pipelines often saturate at 20-40% sustained, which lengthens the cluster's saturation horizon and lowers the marginal cost further.

The migration is therefore *economically straightforward* — the only meaningful costs are the in-house engineering investments listed in §9.2, not inference compute.

Appendix A — Failed Integration Attempts (Roadblocks)

This appendix documents every integration attempt that failed to produce measurements, with the exact failure mode and remediation effort. **The cumulative time and cost of these failures IS the workshop's hidden-tax finding.**

A.1 dots.ocr-1.5 on H200 via vLLM — three-stage failure

Stage 1: vLLM 0.10.1 (released March 2025). Attempted the published incantation `vllm serve rednote-hilab/dots.ocr --trust-remote-code`. Failed with `ValueError: Unrecognized configuration class DotsOCRConfig for AutoModel`. dots.ocr's custom config class is downloaded by `--trust-remote-code` but not registered with `transformers.AutoModel` until you import the model's vLLM-compat shim. The vendor's HF README documents a `sed`-patch of vLLM's CLI entry script to inject the import; we did not pursue this path because it would not survive an RHOAI ServingRuntime container.

Stage 2: vLLM 0.11.0 (released October 2025) — the version the vendor README pins. Same incantation failed with `AttributeError: 'Qwen2VLImageProcessor' object has no attribute 'min_pixels'`. The attribute was removed in `transformers` 5.x; vLLM 0.11.0 was tested against transformers 5.0.x but not 5.8.x (the version pip resolved on our pod).

Stage 3: pinned `transformers==4.55.0` with `--no-deps`. New error: `pydantic_core._pydantic_core.ValidationError: 1 validation error for ModelConfig`. vLLM 0.11.0's internal Pydantic schemas differ between transformers 4.x and 5.x; downgrading transformers triggers a different validation failure.

Pod time burned: ~45 min. **Resolution:** abandoned. dots.ocr cited from vendor numbers (94+ on OmniDocBench v1.5).

Wider lesson: dots.ocr is bleeding-edge enough that `vllm serve <hf_id>` is not reliably "drop-in" across vLLM/transformers minor versions. Production deployment requires either pinning the exact (vLLM, transformers) pair the vendor tested with, or running the vendor's Docker image (`rednotehilab/dots.ocr:vllm-openai-v0.9.1`) which freezes the dependency set.

A.2 DeepSeek-OCR-2 on H200 via vLLM — five-stage failure

Stage 1: First boot failed on missing pip packages: `addict`, `matplotlib` (needed by DeepSeek-OCR's HF modeling code, not declared in the model card requirements).

Stage 2: After installing those: missing `easydict`. Installed.

Stage 3: After installing easydict + `timm` + `einops` + `opencv-python` preemptively: `ImportError: cannot import name 'LlamaFlashAttention2' from 'transformers.models.llama.modeling_llama'`. The class was removed in transformers 4.48+; DeepSeek-OCR's HF modeling code references it directly.

Stage 4: With `--logits-processors vllm.model_executor.models.deepseek_ocr:NGramPerReqLogitsProcessor` (per the vLLM recipes page) to bypass the trust-remote-code path: same ImportError. The vLLM-native deepseek_ocr code path also references the removed class.

Stage 5: Acknowledged that any vLLM 0.11.0 + transformers ≥4.48 combination cannot import DeepSeek-OCR's HF modeling code without a vendored LlamaFlashAttention2 class. Abandoned self-host.

Pod time burned: ~25 min. **Resolution:** pivoted to Novita serverless (`deepseek/deepseek-ocr-2`) which has the integration solved on their side. The first hosted run also failed (degenerate prompt-fragment loop with our English instruction prompt); resolved by switching to the model's native task token `<|grounding|>Convert the document to markdown.` . **Lesson:** OCR-specialist VLMs need their published task tokens, not English prompts.

A.3 PaddleOCR-VL-1.5 — three-platform failure cascade

Platform 1: laptop, Python 3.13, Windows. `pip install paddleocr` fails on the transitive `python-bidi` package — its PEP 517 metadata build does not work on Windows + Python 3.13 (no wheels exist).

Platform 2: H200 RunPod pod, Python 3.11, Linux. `pip install paddlepaddle-gpu==3.2.1 -i https://www.paddlepaddle.org.cn/packages/stable/cu126/` times out — the Baidu CN mirror is unreachable from the RunPod ap-jp-1 region. The default PyPI index has older paddle wheels available, but we did not retest under time pressure.

Platform 3: laptop, Python 3.11 **in a fresh venv, Windows.** `paddlepaddle 3.3.1` and `paddleocr 3.5.0` install cleanly. `PaddleOCRVL()` initialises, downloads the layout-detect model (PP-DocLayoutV3) and the VLM weights (PaddleOCR-VL-1.5-0.9B). Then **Windows fatal exception: access violation (exit code 0xC0000005) at `paddlex/inference/models/common/transformers/transformers/model_utils.py:213` in `_load_part_state_dict_from_safetensors`** during VLM weight load (8 of 18 transformer layers initialised). This is paddle's custom safetensors loader native-crashing on Windows.

Workaround attempt: `vl_rec_backend="transformers"` to bypass paddle's loader and use HuggingFace transformers instead. This required `pip install transformers torch` , which loaded `torch/lib/shm.dll` and triggered `OSError: [WinError 127] The specified procedure could not be found` — a paddle/torch DLL conflict in the same venv on Windows.

Resolution: abandoned local install on Windows. The two-tier production architecture (paddle's layout-detect locally + a hosted VLM endpoint) remains viable; Novita's "on-demand" PaddleOCR-VL deployment requires a custom GPU rental, which is in progress at the time of writing. Published vendor number (94.5% OmniDocBench v1.5) is cited in the matrix.

Wider lesson: paddle's Windows wheels have not been tested at the scale of a full VLM weight load. Linux is the de-facto target. Any production replica using PaddleOCR-VL must run its orchestrator tier in a Linux container — either WSL2 on a developer laptop or a Linux RHOAI pod.

A.4 OCR-specialist VLM prompt sensitivity (DeepSeek-OCR-2 first-run)

After the Novita endpoint came up, the first 8-page run with our standard Markdown+KV instruction prompt produced **complete degenerate output**: empty results on FUNSD and IAM, and 18 KB of repeated prompt fragments (`If the document contains a table, use the 'table' key to identify the table, ...`) on the SEC and DocILE pages. DeepSeek-OCR-2 is a task-token model; English instructions are interpreted as text TO INCLUDE in the OCR output rather than as instructions. Switching to `<|grounding|>Convert the document to markdown.` fixed it.

Wider lesson: OCR-specialist VLMs are **not drop-in replacements for generic VLMs at the prompt level**. The integration cost includes per-model prompt engineering against the model's published task tokens.

Appendix B — Per-Page Per-Stack Detail

The full per-page-per-stack breakdown for the general corpus is committed at `results/scores_local.parquet` (40 rows × 13 axes) and for the feature corpus at `results/feature_scores.parquet` (28 rows). The full per-page raw model outputs are at `results/<stack>/raw/<page_id>.json` for archival inspection.

A condensed view of the general corpus (medians by stack):

Stack	Pages OK	Σ raw text	Σ tables	Σ KV pairs	Median latency
docling	8/8	11 795 chars	11	0	3.24 s
qwen-8b	8/8	10 245 chars	9	36	9.97 s
qwen-30b-a3b	8/8	9 816 chars	9	48	12.09 s
qwen-32b	8/8	14 178 chars	10	114	11.11 s
qwen-235b-a22b	8/8	10 238 chars	9	58	14.53 s
qianfan-ocr	8/8	~30 000 chars (incl. 1 over-long multilingual page)	7	0	9.31 s
deepseek-ocr	8/8	varies (bbox-grounded format, post-parse 8 tables, 12 layout blocks per page)	8	0	4.03 s

---

Appendix C — Repository Structure

```
docintel-benchmark/
├── adapters/                          # one Python module per stack + shared base
│   ├── base.py                        # StackAdapter abstract class
│   ├── schema.py                      # PageResult / Table / KVPair / etc.
│   ├── openai_compatible_base.py      # OpenAI-compat VLM base + table parsers
│   ├── docling_adapter.py             # Docling → PageResult
│   ├── qwen_adapter.py                # Qwen3-VL family
│   ├── baidu_adapter.py               # Qianfan-OCR (free on OpenRouter)
│   ├── deepseek_adapter.py            # DeepSeek-OCR-2 via Novita
│   ├── paddle_vl_adapter.py           # PaddleOCR-VL (in progress, hybrid mode)
│   ├── dots_adapter.py                # dots.ocr (skipped, vendor numbers cited)
│   └── llama_cpp_base.py              # legacy llama-cpp path (kept for reference)
├── corpus/                            # 12 hand-picked pages + gold-truth
│   ├── sec_10k/                       # NVDA + JPM 10-K pages
│   ├── funsd/                         # form
│   ├── cord/                          # receipt
│   ├── fintabnet/                     # table
│   ├── iam/                           # handwriting
│   ├── omnidocbench/                  # multilingual
│   ├── docile/                        # invoice (gitignored — research-only license)
│   └── features/                      # checkboxes / signatures / formulas / codes
├── corpus_meta/corpus_manifest.py     # source of truth for which page tests what
├── metrics/                           # KV F1, CER, table-TEDS-lite, rubric
├── results/                           # per-stack parquet + raw JSON dumps
│   ├── <stack>/raw/<page_id>.json     # raw model output per page
│   ├── <stack>_run_<ts>.parquet       # run-level summary
│   ├── scores_local.parquet           # multi-axis scores
│   └── feature_scores.parquet         # feature-axis scores
├── scripts/                           # CLI entrypoints
│   ├── run_eval.py                    # main bench-off runner
│   ├── run_features.py                # feature-axis runner
│   ├── score_results.py               # general-axis scorer
│   ├── score_features.py              # feature-axis scorer
│   ├── reparse_tables.py              # idempotent table parser back-fill
│   ├── build_feature_corpus.py        # feature corpus builder
│   ├── fetch_sec.py                   # SEC EDGAR 10-K fetcher
│   ├── fetch_hf_datasets.py           # HF dataset puller
│   └── h200_runbook.sh                # RunPod H200 deployment runbook
├── docs/
│   ├── workshop_report.md             # this paper
│   ├── benchmark_spec.md              # the original brief
│   ├── findings.md                    # iterative findings log
│   ├── azure_di_feature_comparison.md  # full feature comparison matrix
│   ├── openshift_ai_deployment.md     # RHOAI deployment notes
│   ├── rubric.md                      # manual rubric for axes without gold
```

```
│   └── measurements_local.csv      # per-stack-per-page measurement CSV
├── requirements.txt                # pinned dependencies
├── .env.example                    # required API keys
├── .gitignore                      # excludes .env, .venv, DocILE corpus, IDE-local files
└── README.md                       # quickstart
```

Appendix D — Side-by-Side Rendered Output Gallery

This appendix shows the **actual extracted output from each stack** on a single representative page (`sec-10k-bank-01` — the JPMorgan Chase consolidated income statement). The same input image is processed by every stack with each stack's chosen prompt; the snippets below are the first ~600 characters of each stack's `raw_text` for direct visual comparison. The full per-page outputs are archived alongside this report.

The contrast between output formats is itself a workshop finding: every stack solves OCR differently, and the choice of stack determines what your downstream pipeline must accept.

D.1 Docling — clean Markdown with proper table structure

```
JPMorgan Chase & Co. Consolidated statements of income

| Year ended December 31, (in millions, except per share data)   | 2025     | 2024     | 2023
|----------------------------------------------------------------|----------|----------|------
| Revenue                                                        |          |          |
| Investment banking fees                                        | $ 9,615  | $ 8,910  | $ 6,5
| Principal transactions                                         | 27,212   | 24,787   | 24,46
| Lending- and deposit-related fees                              | 9,093    | 7,606    | 7,413
| Asset management fees                                          | 20,327   | 17,801   | 15,22
```

Note: Docling produces this from a **structural** pipeline (DocLayNet layout detection → TableFormer cell-level extraction → RapidOCR text recognition), not a generative VLM. Its output is the closest match to "what Azure DI's Layout API would return." 11 tables, 323 cells across the 8-page corpus.

D.2 Qwen3-VL-8B — Markdown with bold revenue subtotal

````
```markdown
# JPMorgan Chase & Co.
## Consolidated statements of income

Year ended December 31, (in millions, except per share data)

|  | 2025 | 2024 | 2023 |
|---|---|---|---|
| **Revenue** |  |  |  |
| Investment banking fees | $ 9,615 | $ 8,910 | $ 6,519 |
| Principal transactions | 27,212 | 24,787 | 24,460 |
````

Note: 8B's output wraps everything in a Markdown code fence (`\ ``markdown`` `) — a benign formatting quirk our raw_text scorer handles. Bold is correctly applied to the "Revenue" header row. 9 tables across the corpus.

D.3 Qwen3-VL-30B-A3B — split-cell artefact on currency-prefixed columns

```
| | 2025 | | 2024 | | 2023 |
| :--- | :--- | :--- | :--- | :--- | :--- |
| **Revenue** | | | | | |
| Investment banking fees | $ | 9,615 | $ | 8,910 | $ | 6,519 |
| Principal transactions | | 27,212 | | 24,787 | | 24,460 |
```

Note: the MoE 30B-A3B variant emits the `$` symbol in its own column on currency-prefixed rows, doubling the column count and breaking the table topology. A prompt-tuning concern; the model's actual reading is correct, just the layout normalization is off. **This is the only Qwen size with this artefact.**

D.4 Qwen3-VL-32B — clean Markdown, bold subtotals (best of the Qwen sizes for 10-K tables)

```
# JPMorgan Chase & Co.
## Consolidated Statements of Income

Year ended December 31, (in millions, except per share data)

| | 2025 | 2024 | 2023 |
|---|---|---|---|
| **Revenue** | | | |
| Investment banking fees | $ 9,615 | $ 8,910 | $ 6,519 |
| Principal transactions | 27,212 | 24,787 | 24,460 |
| ...
| **Noninterest revenue** | **87,004** | **84,973** | **68,837** |
```
```

Note: the only Qwen size that bolds **both** section headers and computed subtotals (Noninterest revenue, Total net revenue, Net income, etc.) in the income statement. Subtotal-bolding is what Azure DI's Layout API does natively. 10 tables across the corpus, the highest of any Qwen size.

D.5 Qwen3-VL-235B-A22B — clean Markdown, no bold subtotals

````
```markdown
# JPMorgan Chase & Co.
## Consolidated statements of income

| Year ended December 31, (in millions, except per share data) | 2025 | 2024 | 2023 |
|--- | --- | --- | ---|
| **Revenue** |  |  |  |
| Investment banking fees | $ 9,615 | $ 8,910 | $ 6,519 |
| Principal transactions | 27,212 | 24,787 | 24,460 |
````

Note: the 235B is more conservative on formatting — bolds only the "Revenue" section header, not subtotals. Higher per-token cost without a corresponding quality gain over the 32B on this page. Across the corpus, 235B extracts **fewer KV pairs (58)** than the 32B (114) — see §5.2 finding #2.

D.6 Baidu Qianfan-OCR-Fast — HTML table format

```
### JPMorgan Chase & Co. Consolidated statements of income

<table><tr><td>Year ended December 31, (in millions, except per share data)</td><td>2025</td><t
```

Note: Qianfan emits **HTML** rather than Markdown — `<table><tr><td>` syntax with proper row markers. Our HTML table parser handles this. **0.972 mean cell-content agreement with Docling under the original prompt** — the highest fidelity of any VLM in the bench-off, and Qianfan is the free option.

D.7 DeepSeek-OCR-2 — bbox-grounded typed-block format

```
sub_title[[65, 52, 401, 89]]
## JPMorgan Chase & Co.
Consolidated statements of income

table[[66, 123, 928, 655]]
<table>Year ended December 31, (in millions, except per share data)<td colspan="2">2025<td col
```

Note: a fundamentally different output shape. Each region is typed (`sub_title` , `text` , `table` , `image` , `list`) with **0–1000 normalized bounding-box coordinates**. Tables are emitted as HTML *without* `<tr>` row markers, which means cell-by-cell alignment for scoring is hard (Appendix B documents the parser limitation). For high-throughput pipelines that consume bbox-grounded output downstream, this is the richest format; for human-readable Markdown, it requires post-processing.
```

D.8 Pyzbar specialist — codes-only output (different page)

For comparison, the `pyzbar` specialist's output on the synthetic codes page (not the JPM 10-K — pyzbar doesn't OCR text):

```
qr :: https://examplecorp.example/order/EXC-2026-05-09
ean13 :: 5901234123457
```

Note: 19 ms latency, perfect 1.000 exact-match. **Drop-in specialist for the orchestration tier — recommended in any production replica that handles barcodes/QR codes.**

Headline takeaway from the gallery

Five different output formats from seven stacks on the same input page — that's the integration surface any orchestration layer must accept. The **recommended hybrid (Docling + Qwen-32B + Qianfan + pyzbar)** intentionally chooses stacks with three complementary output shapes (structural Markdown / Markdown VLM / HTML / decoded payload strings) — each playing the role it's best at, with normalisation happening at the orchestration tier rather than inside any single model.

End of report.